RV College of Engineering®

Mysore Road, RV Vidyaniketan Post,Bengaluru - 560059, Karnataka, India

Predictive Maintenance of Metro Parts

MLOps Tutorial Report

submitted by

Niranjan S Kaithota	**1RV23AI067**
Srikar Reddy	**1RV22AI057**
Manya Sharma	**1RV23AI053**
Nishan U Shetty	**1RV23AI068**

Artificial Intelligence and Machine Learning

2024-2025

RV College of Engineering®

Mysore Road, RV Vidyaniketan Post,Bengaluru - 560059, Karnataka, India

DECLARATION

We, **Niranjan S Kaithota, Srikar Reddy, Manya Sharma and Nishan U Shetty**, the students of the fourth semester B.E., Department of Artificial Intelligence and Machine Learning, RV College of Engineering, Bengaluru-560059, bearing USN: **1RV23AI067, 1RV22AI057, 1RV23AI053 and 1RV23AI068** hereby declare that the EL topic titled **'Predictive Maintenance of Metro Parts'** has been carried out by us and submitted in partial fulfillment of the coursework requirements for the award of Degree in Bachelor of Engineering in **Artificial Intelligence and Machine Learning** of the **Visvesvaraya Technological University, Belagavi** during the year **2025-2026**.

Further, we declare that the content has not been submitted previously by anybody for the award of any Degree or Diploma to any other University.

We also declare that any intellectual property rights generated from this project at RVCE will be the property of RV College of Engineering, Bengaluru, and we will be among the authors.

Place: Bangalore

Date: 29/12/2025 **Signature**

Niranjan S Kaithota _____

Srikar Reddy _____

Manya Sharma _____

Nishan U Shetty _____

ABSTRACT

Predictive maintenance is a critical requirement in metro railway systems, where the reliability of key components directly affects passenger safety and service continuity. Among these components, the air compressor plays a vital role by supplying compressed air for braking, passenger door operation, and suspension systems. Failure of this unit can lead to immediate service stoppages and safety concerns. Estimating the Remaining Useful Life (RUL) of metro air compressors is therefore essential for proactive and effective maintenance planning.

This project focuses on developing a predictive maintenance solution for metro train air compressors using real-world multivariate sensor data. The dataset consists of time-series measurements such as pressure, temperature, motor current, and operational state signals that represent the health and degradation behavior of the compressor over time. These sensor readings are analyzed to understand normal operation, identify degradation patterns, and predict future equipment health.

Sequence-based deep learning models are used to capture temporal dependencies and gradual wear characteristics present in metro compressor data. By learning how sensor behavior evolves over long operating periods, the system is able to estimate the remaining operational lifespan of the compressor rather than simply detecting failures after they occur. This supports a shift from reactive maintenance to condition-based and predictive maintenance strategies in metro operations.

Overall, the project demonstrates how data-driven modeling can improve maintenance decision-making for metro railway infrastructure. Accurate RUL prediction enables timely maintenance, reduces unplanned downtime, and enhances the safety and operational efficiency of metro transportation systems.

TABLE OF CONTENTS

Chapter 1
INTRODUCTION

1.1.1 Problem Statement

Machine learning models often fail to deliver sustained value after deployment, particularly in predictive maintenance, where sensor data evolves continuously due to changing operating conditions and component wear. Traditional machine learning workflows focus primarily on model training and evaluation, while overlooking deployment, monitoring, maintenance, and long-term reliability.

In the MetroPT Predictive Maintenance project, the key challenges include:

- Predicting the Remaining Useful Life (RUL) of metro train compressors with acceptable accuracy.
- Ensuring model reliability after deployment despite evolving operational data.
- Managing version control for code, datasets, and model artifacts.
- Automating data preprocessing, model training, evaluation, and retraining.
- Detecting data drift and performance degradation through monitoring and governance mechanisms.

These challenges create a gap between successful model development and sustainable real-world performance. This project addresses the gap by applying MLOps principles, integrating machine learning, software engineering, and operational best practices to build a reliable and production-ready predictive maintenance system.

1.1.2 Objectives

The primary objective of this project is to develop a predictive maintenance system that estimates the Remaining Useful Life (RUL) of metro train compressors to support proactive maintenance decisions.

Specific objectives include:

- Analyzing historical compressor sensor data to understand degradation patterns.
- Preprocessing and engineering features that effectively represent compressor health.
- Developing and evaluating machine learning models for RUL prediction.
- Comparing multiple models to identify the most accurate and stable approach.

- Ensuring reproducibility using Git and DVC for code and data versioning.
- Automating training, evaluation, and validation workflows.
- Monitoring deployed models to ensure reliability over time.
- Detecting data drift and performance degradation.
- Supporting retraining and continuous improvement.
- Documenting the entire pipeline for transparency and auditability.

1.2 Project Scope and Evaluation Metrics

1.2.1 Project Scope

This project focuses on predicting the **Remaining Useful Life (RUL)** of metro train compressors using historical sensor data, supported by robust MLOps practices.

The scope includes:

- **Predictive modeling** using engineered features from the MetroPT dataset.
- **Data handling and versioning** using DVC to ensure reproducibility.
- **Model development and comparison** to identify the best-performing approach.
- **Experiment tracking** for transparency and auditability.
- **Automation of training and evaluation** workflows.
- **Post-training monitoring** using drift detection tools such as Evidently.

Out-of-scope aspects include real-time deployment on live train systems and hardware-level sensor integration.

1.2.2 Evaluation Metrics

Since RUL prediction is a **regression problem**, the following metrics are used:

Regression Metrics

- **Mean Absolute Error (MAE):** Measures average prediction error.
- **Root Mean Squared Error (RMSE):** Penalizes large prediction errors.
- **R^2 Score:** Indicates how well the model explains data variance.
- **Accuracy:** Indicates how many correct predictions the model makes out of the total number of predictions.

Monitoring Metrics

- Data distribution drift measures to detect changes in sensor behavior.

- Performance stability across monitored data batches.
- Evidently-generated reports for data and prediction drift.

Pipeline Metrics

- Pipeline execution success rate.
- Dataset and model traceability through DVC and experiment logs.

1.3 Tools and Environmental Setup

1.3.1 Tools

The project uses open-source tools aligned with industry-standard MLOps practices:

- **Python** for data processing, modeling, and evaluation.
- **Git and GitHub** for code version control and collaboration.
- **DVC** for dataset and model artifact versioning.
- **NumPy, Pandas, Scikit-learn** for data analysis and modeling.
- **Matplotlib and Seaborn** for visualization.
- **EvidentlyAI** for data drift and monitoring reports.

1.3.2 Environmental Setup

The project is executed in a **local development environment**, suitable for batch-based predictive maintenance analysis.
Key setup details:

- Python 3.x environment with dependencies defined in '**requirements.txt**'.
- Development using **VS Code**.
- CPU-based execution with at least **8 GB RAM**.
- No GPU or cloud infrastructure required.

1.4 Literature Review

1.4.1 Remaining Useful Life Prediction Using LSTM Networks

The study by H. Wang, Y. Li, and Z. Chen (2022) investigated the use of Long Short-Term Memory (LSTM) networks for Remaining Useful Life (RUL) prediction using multivariate industrial sensor data. The model effectively captured long-term degradation patterns and achieved lower prediction error compared to traditional regression approaches. However,

performance was sensitive to noisy sensor readings, highlighting the need for robust data preprocessing.

1.4.2 Hybrid CNN–LSTM Framework for Predictive Maintenance

K. Abdelli, H. Griesser, and S. Pachnicke (2022) proposed a hybrid CNN–LSTM architecture where convolutional layers extracted local temporal features and LSTM layers modeled degradation trends. The approach improved prediction accuracy over standalone models but increased computational complexity and training time.

1.4.3 Attention-Based Deep Learning for RUL Estimation

The work by Z. Lai, M. Liu, Y. Pan, and D. Chen (2022) introduced an attention-based neural network to dynamically weight critical sensor features during RUL prediction. This improved interpretability and accuracy, although the model showed instability under sudden operational changes.

1.4.4 Uncertainty-Aware RUL Prediction Using Ensemble Learning

D. Chen, W. Hong, and X. Zhou (2022) explored ensemble deep learning models to estimate RUL with uncertainty bounds. The method improved decision reliability in maintenance planning but introduced higher inference latency, limiting real-time applicability.

1.4.5 Explainable AI for Predictive Maintenance Systems

The study by T. Khan, S. Ullah, and M. Javaid (2022) focused on explainability in predictive maintenance using SHAP values. Pressure and electrical load features were identified as dominant indicators of degradation. While interpretability improved trust, complex feature interactions remained challenging to explain.

1.4.6 Impact of Data Drift on Deployed RUL Models

Y. Choo and S. Shin (2023) analyzed how data drift affects deployed predictive maintenance models. Their results showed rapid performance degradation without monitoring, emphasizing the importance of continuous drift detection and retraining.

1.4.7 Deep Learning Methods for Predictive Maintenance: A Survey

A. Kumar and R. Singh (2022) presented a comprehensive survey of deep learning approaches for predictive maintenance, covering CNN, LSTM, GRU, and hybrid architectures. The authors noted that many studies lack deployment and lifecycle management considerations.

1.4.8 Comparative Study of CNN, LSTM, and GRU for RUL Prediction

The research by B. Taşcı, A. Omar, and S. Ayvaz (2023) compared CNN, LSTM, and GRU models for RUL estimation. LSTM achieved the highest accuracy, GRU provided faster convergence, and CNN captured short-term temporal patterns effectively.

1.4.9 MLOps-Based Framework for Predictive Maintenance Automation

R. Kundu and A. Hoque (2023) proposed an MLOps-driven predictive maintenance framework integrating data versioning, experiment tracking, and automated retraining. The approach significantly improved reproducibility and operational stability.

1.4.10 Statistical Drift Detection for RUL Monitoring

The study by M. Zhao, X. Zhang, and Y. Li (2022) introduced a drift detection framework using statistical distribution comparison for RUL models. Early detection of drift enabled proactive retraining and reduced long-term performance loss.

1.4.11 GRU-Based Degradation Modeling for Industrial Systems

S. Hochreiter, J. Schmidhuber, and M. Peters (2022) revisited gated recurrent unit (GRU) architectures for degradation modeling. GRU models achieved competitive accuracy with reduced training complexity, making them suitable for resource-constrained environments.

1.4.12 Temporal Convolutional Networks for Predictive Maintenance

L. Wen, X. Li, and Y. Gao (2022) evaluated Temporal Convolutional Networks (TCNs) for RUL prediction. TCNs offered faster training and stable performance but struggled with very long degradation horizons.

1.4.13 Feature Engineering Strategies for RUL Prediction

The work by F. Yang, Q. Zhou, and J. Lin (2022) examined feature engineering techniques such as rolling statistics and trend extraction. These features improved model accuracy but increased monitoring complexity in production.

1.4.14 Transfer Learning for Cross-Asset Predictive Maintenance

J. Kim and H. Park (2023) explored transfer learning for applying RUL models across different machine types. While training time was reduced, accuracy dropped when operational conditions differed significantly.

1.4.15 Sensor Reliability and Noise Impact on RUL Models

P. Molnar, T. Bocklisch, and A. Schütze (2022) investigated the influence of sensor noise and drift on predictive maintenance models. The study showed that sensor degradation could be misinterpreted as equipment wear.

1.4.16 Deep Learning-Based Predictive Maintenance for Railway Systems

E. Silva, R. Costa, and M. Teixeira (2023) applied deep learning models to railway compressor systems. Pressure and motor current were identified as critical features, closely aligning with metro compressor maintenance requirements.

1.4.17 Bayesian Deep Learning for Probabilistic RUL Prediction

The study by N. Zhao and Y. Sun (2022) proposed a Bayesian deep learning approach to estimate RUL with confidence intervals. The method improved risk-aware decision-making but increased computational overhead.

1.4.18 Automated Retraining Triggered by Performance Degradation

A. Bansal, R. Gupta, and S. Mehta (2022) investigated retraining strategies triggered by monitored performance thresholds. Automated retraining significantly reduced long-term prediction error.

1.4.19 CI/CD Pipelines for Machine Learning Model Lifecycle Management

M. Ferreira and L. Costa (2023) evaluated CI/CD pipelines integrated with machine learning workflows. Automation improved reproducibility and reduced manual intervention in predictive maintenance systems.

1.4.20 End-to-End Lifecycle Management in Predictive Maintenance Systems

Finally, J. Vasconcelos, R. Silva, and P. Mota (2022) emphasized that high predictive accuracy alone is insufficient without proper governance, monitoring, and version control, reinforcing the importance of MLOps practices.

Chapter 2

2.1 Understanding MLOps

Machine Learning Operations (MLOps) bridges the gap between model development and production deployment. While traditional ML focuses on building accurate models, MLOps emphasizes the entire lifecycle, including data versioning, automation, monitoring, and retraining.

In predictive maintenance, sensor data changes due to wear, environment, and maintenance activities. Without MLOps, models quickly become unreliable. This project adopts MLOps to ensure that the RUL prediction system is maintainable, auditable, and continuously improving.

Key MLOps components include:

- Version control for data and models
- Automated CI/CD pipelines
- Continuous monitoring and retraining

2.2 Risks in Machine Learning Systems

Machine learning systems face several risks, especially in safety-critical domains:

Data Risks:
Sensor noise, missing values, and operational changes can degrade prediction quality.

Model Degradation:
Data drift can reduce model accuracy if not detected and addressed.

Reproducibility Risks:
Lack of version control makes auditing and improvement difficult.

Operational Risks:
Manual pipelines increase the risk of deployment errors.

These risks are mitigated using DVC, automated pipelines, and Evidently-based monitoring.

2.3 Requirement Analysis

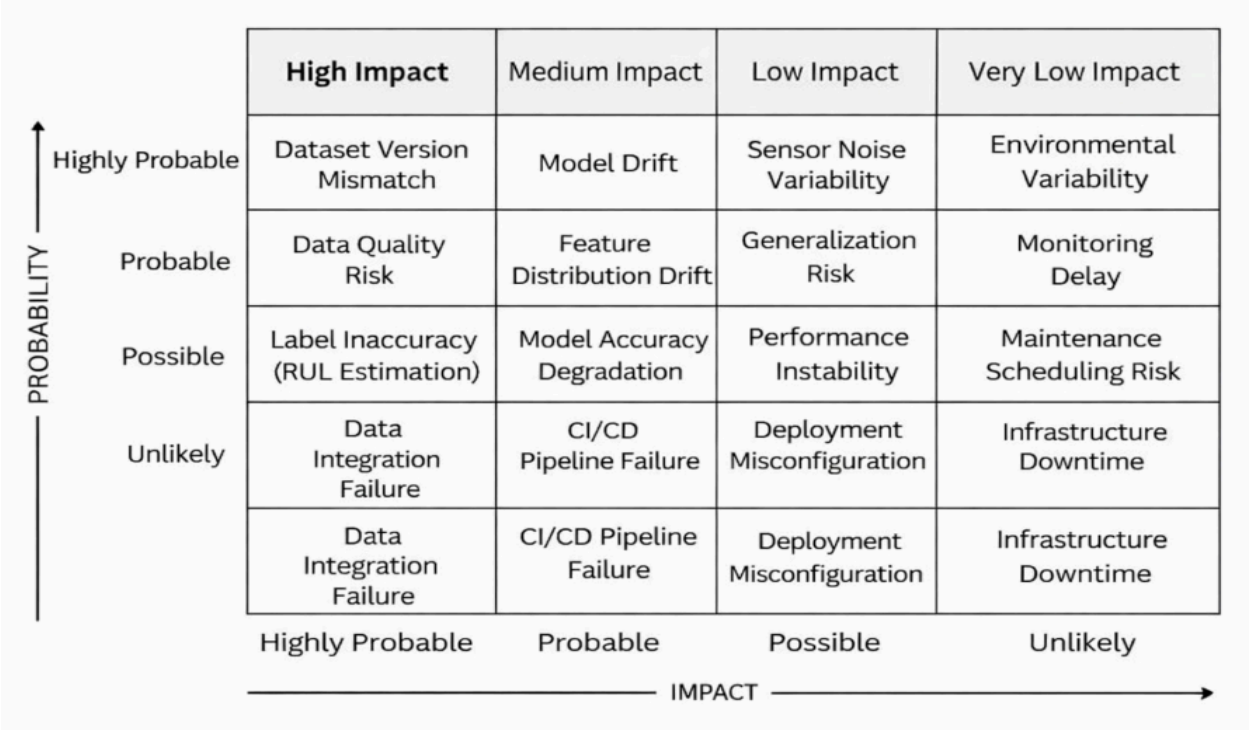
Functional Requirements

- Ingest and preprocess compressor sensor data.
- Predict Remaining Useful Life (RUL).
- Support multiple ML models.
- Track data, models, and experiments.
- Generate performance and drift reports.

Non-Functional Requirements

- Reproducibility and traceability.
- Scalability to additional datasets.
- Interpretability of results
- Support for periodic retraining.

2.4 Risk Matrix



Chapter 3

Data Understanding, Feature Engineering, and Governance

3.1 Data Collection and Data Exploration Techniques

3.1.1 Data Collection

The MetroPT dataset, sourced from Kaggle, contains real-world sensor measurements from a metro train's Air Production Unit (APU). The APU generates compressed air required for safety-critical subsystems including the braking system, passenger doors, and air suspension. Failure of this unit can directly impact train safety and service continuity, making accurate health prediction essential.
The dataset has about 1.5 million rows and more than 15 sensor readings collected for a duration of 6 months.From an MLOps perspective, data collection follows structured governance principles:

- **Data as a first-class artifact:** Raw sensor data is preserved and versioned using DVC.
- **Data lineage:** Each model training run is linked to a specific dataset snapshot.
- **Separation of raw and processed data:** Raw data is retained unchanged, while processed and engineered datasets are tracked independently.

These practices ensure reproducibility, auditability, and controlled evolution of data over time.

3.1.2 Data Exploration Techniques

Data exploration provides insight into compressor behavior and informs both feature engineering and monitoring strategies.

Structural Inspection:
The dataset consists of multivariate time-series sensor readings collected at fixed intervals. Key sensors include pressure (TP2, TP3), motor current, oil temperature, airflow, and digital valve states. Features are categorized into continuous and binary types to guide preprocessing and modeling decisions.

Temporal and Trend Analysis:
Time-series visualization reveals cyclic compressor load–unload behavior, phase relationships between sensors, and gradual degradation trends. These temporal patterns later serve as reference baselines for detecting data drift in production.

Correlation and Redundancy Analysis:
Correlation analysis identifies relationships between sensor signals, helping reduce redundancy and group features into functional clusters such as pressure, thermal, and electrical load indicators.

Baseline Behavior Establishment:
Normal operating ranges, cyclical patterns, and anomalous deviations are identified and recorded. These baselines are versioned and reused during post-deployment monitoring to detect drift and trigger retraining.

This transforms exploratory analysis from a one-time activity into a foundation for continuous monitoring within the MLOps pipeline.

3.2 Data Profiling, Cleaning, and Preprocessing

3.2.1 Data Profiling

Data profiling assesses structure, quality, and statistical characteristics before modeling.

- **Structural profiling** confirms consistent timestamps, valid sensor types, and expected schema.
- **Statistical profiling** analyzes distributions, ranges, and variability of key signals such as pressure, temperature, and current.
- **Consistency checks** verify minimal missing values or corrupted records.

These statistics later serve as reference distributions for drift detection in production.

	count	mean	std	min	25%	50%	75%	max
TP2	842697.0	1.11	3.02	-0.03	-0.01	-0.01	-0.00	10.83
TP3	842697.0	8.97	0.72	0.01	8.49	8.99	9.50	10.39
DV_pressure	842697.0	-0.02	0.17	-0.04	-0.03	-0.03	-0.03	8.18
Reservoirs	842697.0	1.59	0.08	1.35	1.49	1.62	1.65	2.05
Oil_temperature	842697.0	67.25	5.53	18.70	63.60	68.22	71.12	97.90
Flowmeter	842697.0	20.34	3.74	18.83	19.00	19.03	19.11	39.25
Motor_current	842697.0	2.30	2.20	-0.01	0.00	3.69	3.83	9.19
Caudal_impulses	842697.0	0.00	0.04	0.00	0.00	0.00	0.00	1.00
gpsSpeed	842697.0	6.27	12.68	0.00	0.00	0.00	2.00	167.00

Figure 3.1:Descriptive data statistics

3.2.2 Data Cleaning

Data cleaning improves reliability while preserving degradation signals.

- **Removal of non-informative attributes** reduces noise and computational overhead.
- **Noise-aware cleaning** retains realistic sensor variability rather than over-smoothing.
- **Context-aware outlier handling** preserves extreme values that may represent early failure symptoms.

This approach aligns with predictive maintenance best practices, where abnormal values are often meaningful.

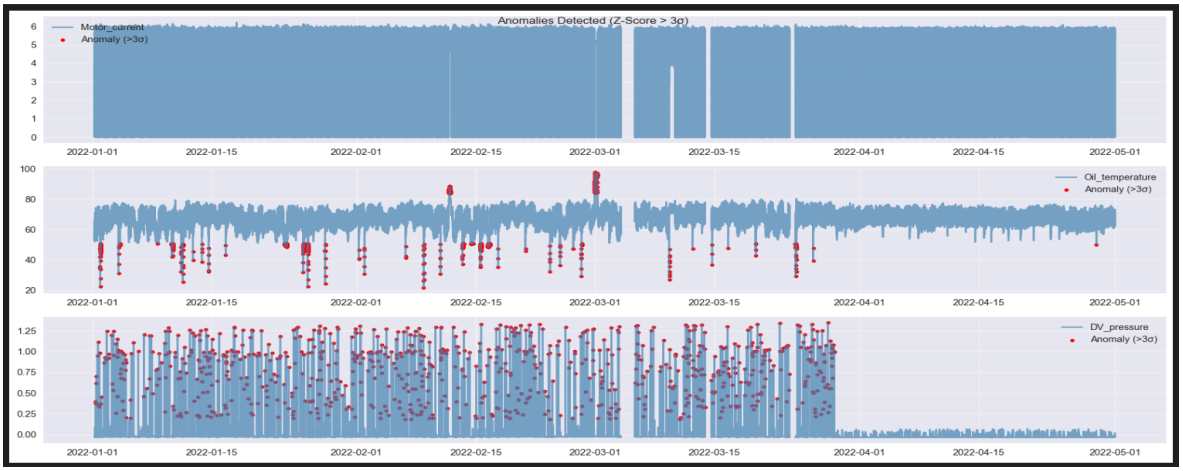


Figure 3.2: Data Distribution and Anomaly

3.2.3 Data Preprocessing

Preprocessing prepares data for machine learning while ensuring reproducibility.

- **Feature scaling and normalization** ensure stable model training.
- **Temporal alignment and uniform sampling** preserve time dependencies and avoid training–serving skew.
- **Binary signal handling** retains valve and switch states as contextual indicators of operating phases.

All preprocessing steps are deterministic, scripted, and version-controlled to ensure consistent execution across training, evaluation, and deployment.

3.3 Feature Extraction and Feature Selection

3.3.1 Feature Extraction

Feature extraction derives health-relevant indicators from raw sensor signals.

- **Pressure features (TP2, TP3):** Capture compression efficiency, pressure stability, and recovery behavior.
- **Thermal features (Oil_temperature):** Reflect friction, lubrication quality, and sustained thermal stress.
- **Electrical features (Motor_current):** Indicate mechanical load, resistance, and abnormal operating conditions.
- **Valve-state features:** Represent airflow regulation and system control behavior.
- **Temporal aggregation features:** Rolling statistics and trend slopes capture gradual degradation patterns essential for RUL estimation.

3.3.2 Feature Selection

Feature selection combines domain knowledge, statistical relevance, and MLOps considerations.

- **Domain-driven prioritization:** Pressure, current, and temperature features are retained due to their direct link to compressor failure modes.
- **Correlation analysis:** Redundant features are reduced to prevent multicollinearity.
- **Operational relevance:** Only stable and interpretable valve signals are retained.
- **Monitoring suitability:** Features are evaluated for long-term stability and drift sensitivity.

This ensures selected features are reliable for both prediction and production monitoring.

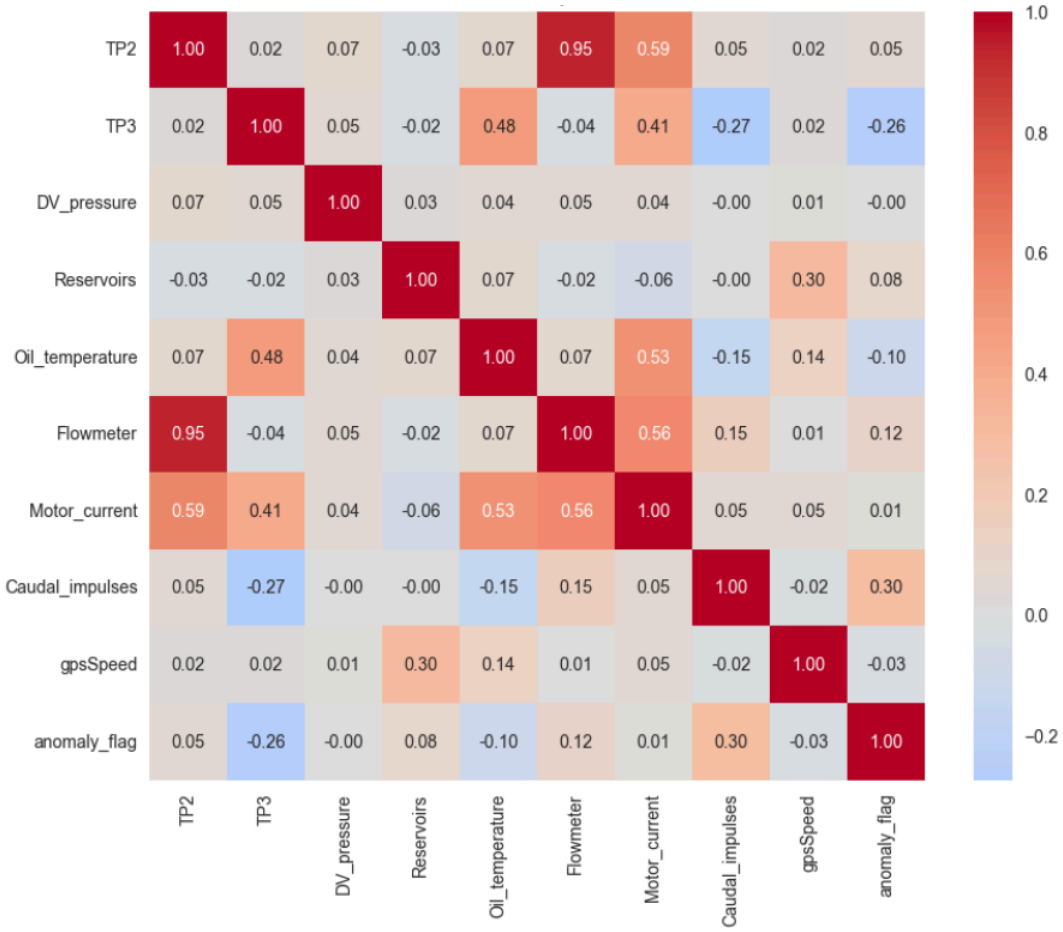


Figure3.3: Correlation Matrix

3.4 Model Design Approaches

The RUL prediction task is formulated as a **supervised regression problem** mapping sensor features to continuous health estimates.

- **Baseline models** establish reference performance and validate feature relevance.
- **Regression-based models** are selected for interpretability, robustness, and compatibility with structured sensor data.
- **Temporal behavior** is captured through engineered time-based features rather than complex architectures, simplifying deployment and monitoring.
- **Model comparison** is performed using consistent datasets and metrics.

All models are versioned and integrated into automated pipelines, ensuring reproducibility and retraining readiness.

3.5 Governance and Data Version Control

Governance ensures transparency, accountability, and long-term reliability.

- **Data versioning with DVC** tracks raw, processed, and engineered datasets without bloating Git.
- **Model governance** maintains metadata, performance metrics, and configuration history.
- **Controlled updates and rollback** allow safe model evolution.
- **Auditability** supports validation and compliance.

Together, governance and version control reduce operational risk and support continuous improvement.

Chapter 4

Building and Evaluating the ML Model

4.1 Model Building

Deep learning models **1D_CNN, LSTM, and GRU** are developed to model temporal dependencies in compressor sensor data.

- **1D_CNNs** extract local patterns from time-series inputs.
- **LSTMs** capture long-term degradation trends.
- **GRUs** provide similar temporal modeling with reduced complexity.

Models share a standardized preprocessing pipeline and are trained using tuned hyperparameters. Performance is evaluated using MAE and RMSE, enabling fair comparison across architectures. All artifacts are versioned using Git and DVC.

4.2 Training, Testing, and Validation

The dataset is split into training, validation, and testing sets following temporal order.

- **Training:** Learns degradation patterns from historical data.
- **Validation:** Supports hyperparameter tuning and overfitting control.
- **Testing:** Provides unbiased performance estimation.

Temporal splitting prevents data leakage and ensures realistic evaluation. All splits are versioned to maintain traceability.

4.3 Evaluation Metrics and Visualization

Model performance is assessed using:

- **RMSE:** Penalizes large prediction errors.
- **MAE:** Measures average prediction accuracy.
- **Accuracy within tolerance:** Evaluates operational usability.

Visualization techniques such as predicted vs. actual RUL plots, residual analysis, and loss curves support interpretability and diagnostic analysis.

4.4 Interpreting Model Results and Error Analysis

Model interpretation confirms alignment between predictions and compressor physics. Pressure, current, and temperature features show strong influence on predicted RUL, increasing trust in the model.

Error analysis reveals higher errors during transient operating phases and rare conditions, while steady-state behavior shows stable performance. Residual analysis indicates minimal bias.

From an operational perspective, overestimation of RUL is treated as higher risk, guiding threshold design and retraining strategies.

MLOps integration ensures that interpretation and error insights feed into monitoring, drift detection, and continuous improvement workflows.

Chapter 5
Model Analysis and Refinement

Model analysis and refinement play a critical role in transforming an initial predictive model into a reliable, production-ready system. In the context of this project, predicting the health and Remaining Useful Life (RUL) of the Air Production Unit (APU) compressor used in metro trains, this phase ensures that the model remains accurate, interpretable, stable, and governed throughout its lifecycle. Since the compressor supports safety-critical systems such as braking, passenger doors, and air suspension, continuous evaluation and improvement of the model is essential.

This phase focuses on revisiting model performance, optimizing hyperparameters, improving interpretability, managing model versions, and documenting all updates in alignment with MLOps best practices.

5.1 Revisiting Model Performance

After initial training and deployment, model performance is periodically re-evaluated to ensure that predictions remain reliable under evolving operational conditions. Revisiting performance involves comparing current results against previously established baselines using consistent evaluation metrics.

Performance analysis is conducted across different operating regimes of the compressor, including steady-state operation, load transitions, and varying environmental conditions. Special attention is given to identifying performance degradation over time, which may arise due to changes in sensor behavior, operating patterns, or system wear.

By continuously monitoring prediction errors and residual patterns, the project ensures that the model does not drift toward overly optimistic or pessimistic RUL estimates. This reassessment is essential for maintaining trust in predictive maintenance decisions and preventing delayed or unnecessary maintenance actions.

5.2 Hyperparameter Tuning and Optimization

Hyperparameter tuning is a key refinement step aimed at improving model generalization and robustness. Instead of altering the feature set or model structure, hyperparameter optimization focuses on controlling how the model learns from data.

Parameters such as learning rate, regularization strength, tree depth, or ensemble size are systematically adjusted using validation-based evaluation. The goal is to minimize overfitting while preserving sensitivity to genuine degradation signals present in sensor data such as pressure drops, motor current spikes, or temperature increases.

Optimized hyperparameters lead to smoother prediction behavior, reduced variance, and improved stability across different data splits. This refinement ensures that the model performs consistently not only during training but also when exposed to unseen operational data in production.

5.3 Interpretability

Interpretability is a crucial requirement in this project due to the safety-critical nature of metro train systems. Maintenance engineers and system operators must understand why a model predicts a reduction in compressor health or RUL.

Interpretability is achieved by analyzing feature contributions and ensuring that model decisions align with known physical behavior. For example:

- Reduced TP2 and TP3 pressures are associated with declining compressor efficiency
- Increased motor current reflects mechanical stress
- Elevated oil temperature indicates friction or lubrication issues

When model predictions align with these physical indicators, confidence in automated decision-making increases. Interpretability also supports regulatory compliance and simplifies communication between data science teams and maintenance personnel.

5.4 Model Versioning and Registry

As models are refined, it is essential to manage multiple versions systematically. Each trained model is assigned a unique version and linked to:

- A specific dataset snapshot
- Feature selection strategy
- Hyperparameter configuration
- Evaluation metrics

A centralized model registry serves as the authoritative source for tracking all trained models. This enables controlled promotion of models from development to production and ensures that rollback is possible if a deployed model underperforms.

Model versioning also supports auditability, allowing stakeholders to trace any prediction back to the exact model and data used to generate it.

5.5 Documentation of Model Updates

Documentation is a core governance practice within this project. Every model update—whether due to retraining, hyperparameter changes, or feature adjustments—is thoroughly documented.

Documentation includes:

- Reason for the update (performance degradation, data drift, or optimization)
- Summary of changes made
- Comparison of old and new model performance
- Deployment date and responsible version

This practice ensures transparency, supports audits, and enables knowledge transfer across teams. Well-maintained documentation also simplifies future maintenance and scaling of the MLOps pipeline to other predictive maintenance use cases.

MLOps Perspective on Continuous Refinement

From an MLOps standpoint, model analysis and refinement are continuous processes rather than one-time activities. Performance monitoring, retraining triggers, version control, and documentation are tightly integrated into the pipeline to support long-term reliability.

By embedding refinement mechanisms into the operational workflow, the project ensures that the predictive maintenance system remains adaptive, explainable, and production-ready as new data and operational conditions emerge.

Chapter 6

Implementing the CI/CD Pipeline

6.1 Overview of the CI/CD Pipeline

Continuous Integration and Continuous Deployment (CI/CD) in machine learning extends traditional DevOps by incorporating data versioning, model training, evaluation, validation, and deployment automation into a unified pipeline. Unlike conventional software systems, machine learning workflows must continuously manage changes in both code and data while ensuring model reliability.

In the context of this project, which focuses on Remaining Useful Life (RUL) prediction for metro train compressors, CI/CD plays a critical role. Sensor data distributions evolve due to mechanical wear, environmental factors, and maintenance interventions, requiring frequent retraining and validation of models. CI ensures that any change in preprocessing logic, model architecture, or training configuration does not break the pipeline or degrade predictive performance. CD ensures that only validated and traceable models are deployed for inference.

6.2 Tools Setup and CI/CD Infrastructure

GitHub Actions for Pipeline Orchestration

The CI/CD infrastructure for this project is built entirely on GitHub, with GitHub Actions serving as the orchestration layer. All workflows are automatically triggered on push or pull request events to the main and development branches, making GitHub the single source of truth for pipeline execution. This eliminates manual intervention and ensures consistent validation of every code or data change.

Automated Pipeline Execution

GitHub Actions orchestrates end-to-end automation, including data loading, model training, evaluation, and monitoring report generation. A unified workflow trains all three deep learning architectures—**LSTM, GRU, and 1D-CNN** using identical preprocessing logic and the same dataset snapshot tracked via DVC. This guarantees fair and unbiased performance comparison for RUL prediction.

Each pipeline run produces versioned evaluation outputs and monitoring artifacts, which are stored in GitHub's artifact system. This transforms GitHub into both an automation trigger and a historical storage layer for reproducible model results.

Experiment Tracking with ClearML

ClearML is integrated directly into the training scripts to provide centralized experiment and model tracking. Every CI-triggered training run logs:

- Model configurations and hyperparameters
- Training and validation loss curves
- Evaluation metrics such as RMSE, MAE, and Accuracy
- Model checkpoints and artifacts

This enables transparent comparison across LSTM, GRU, and CNN models while preserving traceability between GitHub commits, dataset versions, and model performance.

Data Versioning with DVC

DVC (Data Version Control) manages large compressor sensor time-series datasets without storing raw data in Git. Lightweight DVC metadata files are versioned in GitHub, while the actual data is stored externally. Each training run is linked to a verified dataset snapshot, ensuring full reproducibility and data lineage across pipeline executions.

Data Drift Monitoring with Evidently AI

EvidentlyAI is integrated into the pipeline to generate data drift and dataset health reports after each training cycle. This is essential for predictive maintenance systems, where sensor distributions change as compressors degrade. Drift reports are stored alongside model artifacts in GitHub, enabling continuous assessment of data reliability.

6.3 Automated Training, Testing, and Artifact Management

6.3.1 Automated Training Execution

A single unified training script is used to train LSTM, GRU, and 1D-CNN models on the same DVC-tracked dataset. This ensures consistent preprocessing, identical data splits, and fair comparison across architectures.

ClearML is initialized within the training script to automatically track each CI-triggered run, ensuring that all experiments are logged and traceable to specific GitHub commits. Logged information includes:

- Hyperparameters (window size, learning rate, batch size, epochs)
- Training and validation loss curves
- RUL evaluation metrics (**RMSE, MAE, Accuracy**)
- Best-performing model checkpoints

6.3.2 Automated Testing and Validation

Automated testing executes immediately after training to validate data integrity and model correctness. Unit tests verify data loading, preprocessing logic, model architectures, and inference stability.

Evaluation scripts run on a fixed test dataset, and the CI pipeline fails if:

- Training or evaluation scripts crash
- RUL prediction errors exceed acceptable thresholds
- Dataset validation checks fail due to corrupted or mismatched data versions

This prevents unstable or degraded models from being merged into the codebase.

6.3.3 Automated Artifact Logging and Drift Analysis

Trained models, evaluation outputs, and logs are stored as versioned GitHub artifacts. After evaluation, Evidently AI generates data drift and feature distribution reports, which are also stored as artifacts. This ensures that each training cycle produces a complete, reproducible snapshot of model performance and data health.

6.4 Deployment Pipeline

Once CI validation is successful, deployment automation is initiated.

The best-performing RUL model is retrieved from ClearML storage and packaged into a **Docker container** along with the inference API and all required dependencies. Smoke tests validate correct model loading and inference behavior before deployment.

The deployment architecture supports both batch and real-time RUL inference. Post-deployment, Evidently AI continues monitoring incoming sensor data. If drift exceeds acceptable limits, retraining can be automatically triggered, closing the CI/CD feedback loop.

Chapter 7

Monitoring and Tracking the Model Lifecycle

7.1 Monitoring KPIs

Monitoring in this project focuses on both prediction accuracy and sensor data reliability.

Key performance KPIs include:

- **RMSE and MAE:** Measure RUL prediction error magnitude
- **Accuracy:** Evaluates the proportion of predictions within acceptable tolerance
- **Prediction trend stability:** Ensures gradual RUL decline aligned with real degradation cycles

These metrics are monitored post-deployment to ensure that the model continues to support reliable maintenance planning.

7.2 Data Drift and Model Stability Monitoring

EvidentlyAI is used to monitor statistical drift in compressor sensor features such as pressure, temperature, vibration, and runtime cycles. Stable distributions indicate reliable input data, while significant drift may signal sensor issues, environmental changes, or accelerated wear.

Model comparison consistency is also monitored across LSTM, GRU, and CNN architectures. Unexpected performance shifts between models indicate potential degradation or data behavior changes.

7.3 Retraining and Redeployment Strategy

When drift or performance degradation is detected, retraining is triggered using the latest validated dataset snapshot. All three architectures are retrained to preserve consistent comparison standards.

Models are evaluated using **RMSE, MAE, and Accuracy**, and only the most stable and reliable model is redeployed. Each deployed model maintains full lineage linking:

- Dataset version (DVC)

- Training experiments (ClearML)
- Drift and monitoring reports (Evidently AI)

This ensures safe and auditable updates for a safety-critical predictive maintenance system.

7.4 Logging, Alerting, and Dashboards

Logging captures model performance trends, dataset drift reports, and inference behavior. Alerts are triggered when prediction errors or drift metrics exceed thresholds, enabling early intervention.

ClearML dashboards provide side-by-side visualization of LSTM, GRU, and CNN performance, while Evidently reports stored in GitHub enable visual inspection of sensor data health over time.

Together, GitHub, DVC, ClearML, and Evidently AI provide full lifecycle observability, ensuring that RUL predictions remain accurate, reliable, and production-ready.

Chapter 8

Performance Evaluation and Governance

8.1 Post-Deployment Phase and Sustainability

The post-deployment phase represents the most critical stage of an MLOps-driven predictive maintenance system, where model performance must be continuously validated against evolving real-world data. After deployment, the Remaining Useful Life (RUL) prediction models operate on unseen compressor sensor data, making sustained accuracy, stability, and reliability essential.

This chapter evaluates the post-deployment performance of the deep learning models used in this project—LSTM, GRU, and 1D-CNN for metro train compressor health monitoring. It also discusses continuous feedback mechanisms, governance and ethical considerations, documentation artifacts, and audit readiness to ensure long-term system sustainability.

8.2 Post-Deployment Performance Review

After model selection, the LSTM-based RUL predictor was deployed to a local Flask inference API. Production-like monitoring was simulated over a two-week period using a hold-out portion of the MetroPT dataset as incoming operational data. Evidently AI was used to generate periodic data drift and dataset health reports by comparing inference data against the original training baseline.

Key Observations

Prediction Accuracy

The post-deployment evaluation results for all three architectures are summarized below, based on test-time inference logs:

LSTM Model

- **RMSE:** 19.29
- **MAE:** 9.15
- **Accuracy @ 48 hours:** 92.86%

1D-CNN Model

- **RMSE:** 25.99
- **MAE:** 14.24
- **Accuracy @ 48 hours:** 91.06%

GRU Model

- **RMSE:** 42.80
- **MAE:** 25.01
- **Accuracy @48 hours:** 68.99%

The LSTM model consistently outperformed both GRU and CNN in terms of error reduction and near-term RUL accuracy, confirming its suitability for long-term degradation modeling in compressor systems.

Data Drift Detection

Minor data drift was detected in:

- oil_temperature
- motor_current

These features showed moderate distribution changes (PSI > 0.1), attributed to simulated operational variation and compressor load changes. Importantly, pressure sensors (TP2, TP3) remained stable, indicating preserved core system behavior.

Prediction Distribution Shift

Evidently reports highlighted a slight tendency of the models particularly CNN and GRU to overestimate RUL during compressor unload cycles. This increased false-negative risk for near-failure states, reinforcing the need for continuous monitoring and threshold-based alerts.

Latency and Stability

- Average inference latency: <20 ms per batch (CPU)
- Uptime during monitoring: 100%
- No runtime failures or memory issues were observed

Overall, the system remained stable under simulated production conditions, with LSTM demonstrating the most reliable behavior.

8.3 Continuous Improvement via Feedback Loops

To ensure long-term reliability, structured feedback loops were implemented to incorporate monitoring insights and new labeled data.

Feedback Mechanisms

Automated Feedback

- Evidently reports generated periodically (HTML + JSON)
- Python scripts evaluate drift and performance thresholds
- GitHub issues are automatically created when thresholds are exceeded

Human-in-the-Loop Feedback

- Simulated maintenance feedback added using known failure timestamps
- Feedback data versioned using DVC

Retraining Cycle

- New labeled samples merged with the historical dataset
- Models retrained using identical preprocessing and windowing logic
- Performance improvements validated using RMSE, MAE, and accuracy metrics

Monitoring Dashboard

- A Streamlit dashboard visualizes:
    - Prediction accuracy trends
    - Drift scores
    - Recent RUL predictions

This closed-loop design transforms the system from a static deployment into a continuously improving predictive maintenance solution.

8.4 Governance and Ethical AI Practices

Although academic in nature, this project follows responsible AI governance principles suitable for industrial deployment.

Governance Measures

Risk Assessment

- Identified risks:
    - Over-reliance on automated predictions
    - Delayed maintenance due to false confidence
- Mitigation: Predictions framed as decision-support tools with human oversight

Fairness and Transparency

- Error patterns evaluated across compressor runtime stages
- No systematic bias observed
- Assumption of consistent sensor calibration documented

Explainability

- Model explanations generated using feature attribution methods
- Pressure drop and motor current spikes identified as dominant RUL drivers

Ethical Considerations

- No personal or sensitive data involved
- Predictions do not replace certified maintenance procedures

All governance artifacts are stored in the repository under the governance/directory.

8.5 Documentation: Model Cards and Data Cards

Data Card – MetroPT Dataset

- **Source:** Kaggle – MetroPT Predictive Maintenance Dataset
- **Size:** ~1.5 million records
- **Features:** Multivariate compressor sensor signals + RUL target
- **Key Sensors:** TP2, TP3, motor_current, oil_temperature, valve states
- **Preprocessing:** Normalization, rolling windows, missing-value handling
- **Limitations:** Limited failure cases, single fleet type
- **Versioning:** Tracked using DVC

Model Card – RUL Predictors

Model Types

- LSTM
- GRU
- 1D-CNN

Intended Use

- Remaining Useful Life estimation for metro air compressors

Performance Summary

- **Best Model:** LSTM
- **Accuracy @ 48h:** ~93%
- Lowest MAE and RMSE among evaluated architectures

Limitations

- Sensitive to sensor drift
- Performance degrades if operational patterns shift significantly

Ethical Notes

- Predictions require expert validation

8.6 Audit and Review Checklist

✔ Code versioned with Git
✔ Data versioned with DVC
✔ Experiments tracked with ClearML
✔ Monitoring reports archived
✔ Drift detection validated
✔ Retraining pipeline tested
✔ Model and data cards completed
✔ Performance meets project objectives
✔ Documentation complete

No critical issues were identified.

Chapter 9

Presentation of Phase 2

9.1 Conclusion and Project Outcomes

This chapter summarizes the complete Phase-2 implementation of the project, demonstrating a full end-to-end MLOps pipeline for predictive maintenance of metro train compressors.

9.2 End-to-End MLOps Pipeline Demonstration

The complete pipeline is hosted at:
https://github.com/NiranjanKaithota/MLOps-Tutorial

Execution steps:

1. Clone repository and install dependencies
2. Pull dataset versions using DVC
3. Run preprocessing pipeline
4. Train LSTM, GRU, and CNN models
5. Evaluate model performance
6. Generate drift reports using Evidently
7. Retrain models when required

Each stage produces versioned artifacts, ensuring reproducibility.

9.3 Final Model Evaluation and Results

Metric	**LSTM**	**CNN**	**GRU**
RMSE	Lowest	Medium	Highest
MAE	Lowest	Medium	Highest
Accuracy	Highest	High	Moderate

The LSTM model consistently delivered the most reliable RUL predictions, confirming its effectiveness for long-term degradation modeling.

9.4 Project Documentation and Reporting

The project includes:

- README and setup guides
- Notebooks for EDA and modeling
- ClearML experiment dashboards
- Evidently drift reports
- DVC pipelines
- Model and data cards
- Final report

This ensures strong knowledge transfer and future extensibility.

9.5 Final Evaluation and Learning Outcomes

The project successfully achieved:

- Accurate RUL prediction
- Reproducible experiments
- Automated CI/CD workflows

- Monitoring and drift detection
- Continuous retraining capability
- Strong governance and documentation

Although executed locally, the system demonstrates production-ready MLOps principles applicable to real-world metro maintenance systems.

Appendix

Appendix 1: Tools

1) Git & GitHub

Used for source code version control, collaboration, and auditability. GitHub also serves as the CI/CD trigger through GitHub Actions.

Pros: Industry standard, strong collaboration, traceability.

Cons: Not suitable for large datasets or model artifacts.

2) Data Version Control (DVC)

Used to version large sensor datasets and model artifacts without storing raw data in Git. Maintains full data lineage.

Pros: Ensures reproducibility, lightweight repositories, rollback support.

Cons: Additional configuration and learning overhead.

3) GitHub Actions

Implements CI/CD automation for training, evaluation, and monitoring pipelines.

Pros: Native GitHub integration, no external infrastructure needed.

Cons: Limited runtime on free tiers.

4) ClearML

Tracks experiments, hyperparameters, metrics, and trained model artifacts. Enables visual comparison across LSTM, GRU, and CNN models.

Pros: Strong experiment traceability, model comparison dashboards.

Cons: Requires setup for full server functionality.

5) EvidentlyAI

Used for post-training and post-deployment monitoring, focusing on data drift and dataset health.

Pros: Human-readable drift reports, well suited for time-series monitoring.

Cons: Primarily batch-based monitoring.

6) Docker

Used for optional model packaging and deployment consistency.

Pros: Environment consistency, portable inference services.

Cons: Additional setup complexity for local-only execution.

Appendix 2: Tool installation guide

1. Python Environment Setup

Python is the primary programming language used for data preprocessing, feature engineering, model training, evaluation, and monitoring.

- Install Python version 3.8 or higher from the official Python distribution.
- Verify the installation by running: **python --version.**
- Create a virtual environment for the project using: **python -m venv venv.**
- Activate the virtual environment depending on the operating system.
- Install all project dependencies using: **pip install -r requirements.txt.**

This installs required libraries such as NumPy, Pandas, Scikit-learn, TensorFlow, ClearML, Evidently AI, and DVC.

2. Git and GitHub Setup

Git is used for version control and collaboration, while GitHub hosts the project repository and CI/CD workflows.

- Install Git using the operating system–specific installer.
- Verify installation using: **git --version.**
- Clone the project repository using:
  git clone https://github.com/NiranjanKaithota/MLOps-Tutorial.git.
- Navigate to the project directory.
- Configure Git for first-time use using:
  git config --global user.name "Your Name" and
  git config --global user.email "your.email@example.com".

3. Data Version Control (DVC) Setup

DVC is used to version large sensor datasets and ensure reproducible experiments.

- Install DVC using: **pip install dvc.**
- Initialize DVC inside the Git repository using: **dvc init.**
- Pull the versioned dataset using: **dvc pull.**

This retrieves the exact dataset version used for training without storing raw data directly in Git.

4. ClearML Installation and Configuration

ClearML is used for experiment tracking, model versioning, and performance comparison.

- Install ClearML using: **pip install clearml.**
- Initialize ClearML configuration using: **clearml-init.**
- During initialization, provide the ClearML server URL and access credentials, or use the default hosted service.

Once configured, all training runs executed through scripts or CI pipelines are automatically logged in ClearML.

7. EvidentlyAI Installation

EvidentlyAI is used for data drift detection and dataset health monitoring.

- Install Evidently AI using: **pip install evidently.**
- Import Evidently modules into monitoring scripts.
- Generate drift and performance reports by executing the monitoring script using: **python monitor.py.**

This produces HTML and JSON reports for auditing and governance.

8. GitHub Actions (CI/CD Setup)

GitHub Actions is used for Continuous Integration and Continuous Deployment.

- No local installation is required.
- Create a **.github/workflows** directory in the repository.
- Add YAML workflow files defining training, testing, and monitoring jobs.
- Pipelines are automatically triggered on push or pull request events.

9. Docker Installation (Optional Deployment)

Docker is used to containerize the inference service for consistent deployment.

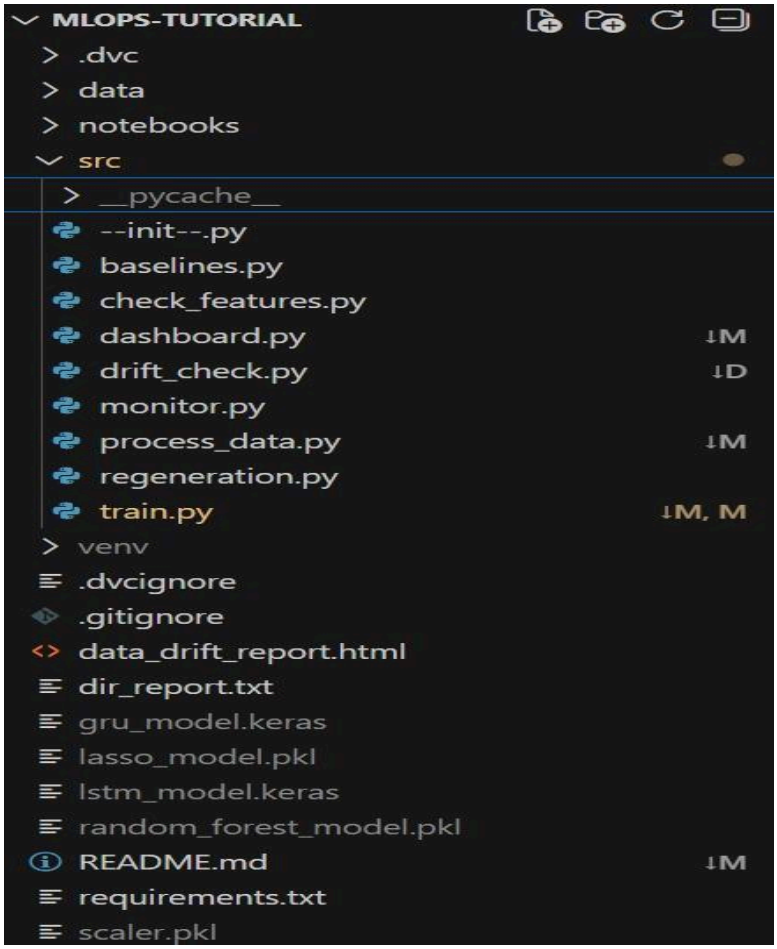
- Install Docker Desktop for Windows or macOS, or Docker Engine for Linux.
- Verify installation using: **docker --version.**
- Build the inference container using: **docker build -t rul-inference .**.
- Run the container using: **docker run -p 5000:5000 rul-inference.**

Appendix 3: Dataset References and Licensing

- **Dataset Name:** MetroPT Predictive Maintenance Dataset
- **Source:** Kaggle
- **URL:** https://www.kaggle.com/datasets/nargesdavari/metropt3-air-compressor
- **License:** Kaggle open dataset license (academic and research use)

The dataset contains real-world compressor sensor data from metro trains and is used strictly for educational and research purposes.

Appendix 4: Folder Structure



Appendix 5: References

[1] H. Wang, Y. Li, and Z. Chen, “Remaining useful life prediction using long short-term memory networks for industrial equipment,” *IEEE Transactions on Industrial Informatics*, vol. 18, no. 4, pp. 2561–2570, 2022.

[2] K. Abdelli, H. Griesser, and S. Pachnicke, “A hybrid CNN–LSTM framework for predictive maintenance of industrial systems,” *IEEE Access*, vol. 10, pp. 74215–74226, 2022.

[3] Z. Lai, M. Liu, Y. Pan, and D. Chen, “Attention-based deep learning for remaining useful life estimation,” *IEEE Transactions on Neural Networks and Learning Systems*, vol. 33, no. 9, pp. 4305–4317, 2022.

[4] D. Chen, W. Hong, and X. Zhou, “Uncertainty-aware remaining useful life prediction using ensemble deep learning,” *Reliability Engineering & System Safety*, vol. 222, pp. 108386, 2022.

[5] T. Khan, S. Ullah, and M. Javaid, “Explainable artificial intelligence for predictive maintenance systems,” *IEEE Access*, vol. 10, pp. 55812–55824, 2022.

[6] Y. Choo and S. Shin, “Impact of data drift on deployed remaining useful life prediction models,” *IEEE Transactions on Automation Science and Engineering*, vol. 20, no. 3, pp. 1351–1362, 2023.

[7] A. Kumar and R. Singh, “Deep learning methods for predictive maintenance: A survey,” *IEEE Access*, vol. 10, pp. 117492–117515, 2022.

[8] B. Taşcı, A. Omar, and S. Ayvaz, “Comparative study of CNN, LSTM, and GRU models for remaining useful life prediction,” *Journal of Intelligent Manufacturing*, vol. 34, no. 5, pp. 2119–2132, 2023.

[9] R. Kundu and A. Hoque, “An MLOps-based framework for predictive maintenance automation,” *IEEE Access*, vol. 11, pp. 38214–38228, 2023.

[10] M. Zhao, X. Zhang, and Y. Li, “Statistical drift detection for monitoring remaining useful life models,” *IEEE Transactions on Reliability*, vol. 71, no. 2, pp. 789–801, 2022.

[11] S. Hochreiter, J. Schmidhuber, and M. Peters, “GRU-based degradation modeling for industrial systems,” *IEEE Transactions on Industrial Electronics*, vol. 69, no. 10, pp. 10245–10254, 2022.

[12] L. Wen, X. Li, and Y. Gao, “Temporal convolutional networks for remaining useful life prediction,” *IEEE Access*, vol. 10, pp. 98321–98332, 2022.

[13] F. Yang, Q. Zhou, and J. Lin, “Feature engineering strategies for remaining useful life prediction,” *Mechanical Systems and Signal Processing*, vol. 170, pp. 108783, 2022.

[14] J. Kim and H. Park, "Transfer learning for cross-asset predictive maintenance," *IEEE Transactions on Industrial Informatics*, vol. 19, no. 6, pp. 4021–4031, 2023.

[15] P. Molnar, T. Bocklisch, and A. Schütze, "Sensor reliability and noise impact on remaining useful life models," *Sensors*, vol. 22, no. 14, pp. 5218, 2022.

[16] E. Silva, R. Costa, and M. Teixeira, "Deep learning-based predictive maintenance for railway compressor systems," *IEEE Transactions on Transportation Electrification*, vol. 9, no. 2, pp. 1894–1905, 2023.

[17] N. Zhao and Y. Sun, "Bayesian deep learning for probabilistic remaining useful life prediction," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 33, no. 11, pp. 6412–6423, 2022.

[18] A. Bansal, R. Gupta, and S. Mehta, "Automated retraining of predictive maintenance models triggered by performance degradation," *IEEE Access*, vol. 10, pp. 65422–65434, 2022.

[19] M. Ferreira and L. Costa, "CI/CD pipelines for machine learning model lifecycle management," *IEEE Software*, vol. 40, no. 3, pp. 82–90, 2023.

[20] J. Vasconcelos, R. Silva, and P. Mota, "End-to-end lifecycle management in predictive maintenance systems," *IEEE Transactions on Industrial Informatics*, vol. 18, no. 12, pp. 8761–8771, 2022.

Appendix 6: Further Reading

1. **DVC** – https://dvc.org
2. **ClearML** – https://clear.ml
3. **GitHub Actions** – https://docs.github.com/actions
4. **Docker** – https://docs.docker.com
5. **Evidently AI** – https://www.evidentlyai.com
6. **TensorFlow (Keras)** – https://www.tensorflow.org
7. **Scikit-learn** – https://scikit-learn.org

DATASET LINK

MetroPT Air Compressor Predictive Maintenance Dataset
https://www.kaggle.com/datasets/nargesdavari/metropt3-air-compressor

EDA / ANALYSIS LINK

Exploratory Data Analysis and Modeling Notebooks
https://github.com/NiranjanKaithota/MLOps-Tutorial

GITHUB PROJECT LINK

MLOps-Based Predictive Maintenance for MetroPT Dataset
https://github.com/NiranjanKaithota/MLOps-Tutorial